

# Creating a VPC using Terraform and Configuring EC2 using Ansible

Aditya Kumar Singh  
RA2311003010916

```
Administrator: Windows PowerShell

PS C:\WINDOWS\system32> choco install terraform -y
Chocolatey v2.4.2
Installing the following packages:
terraform
By installing, you accept licenses for the packages.
terraform v1.12.2 already installed.
Use --force to reinstall, specify a version to install, or try upgrade.

Chocolatey installed 0/1 packages.
See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).

Warnings:
- terraform - terraform v1.12.2 already installed.
Use --force to reinstall, specify a version to install, or try upgrade.
PS C:\WINDOWS\system32> terraform --version
>>
Terraform v1.12.2
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.0. You can update by downloading from https://developer.hashicorp.com/terraform/install

PS C:\WINDOWS\system32> choco install awscli -y
>>
Chocolatey v2.4.2
Installing the following packages:
awscli
By installing, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'
Progress: Downloading awscli 2.36.33... 100%

awscli v2.36.33 [Approved]
awscli package files install completed. Performing other installation steps.
Downloading awscli 64 bit
from 'https://awscli.amazonaws.com/AWSCLIV2-2.36.33.msi'
Progress: 100% - Completed download of C:\Users\ADITYA\AppData\Local\Temp\chocolatey\awscli\2.36.33\AWSCLIV2-2.36.33.msi (47.13 MB).
Download of AWSCLIV2-2.36.33.msi (47.13 MB) completed.
Hashes match.
Installing awscli...
awscli has been installed.
awscli may be able to be automatically uninstalled.
Environment Vars (like PATH) have changed. Close/reopen your shell to
see the changes (or in powershell/cmd.exe just type 'refreshenv').
The install of awscli was successful.
Software installed as 'MSI', install location is likely default.

Chocolatey installed 1/1 packages.
See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).
```

```

PS C:\WINDOWS\system32> pip install ansible
>
Collecting ansible
  Downloading ansible-14.3.1-py3-none-any.whl.metadata (8.1 kB)
Collecting ansible-core==2.21.3 (from ansible)
  Downloading ansible_core-2.21.3-py3-none-any.whl.metadata (7.8 kB)
Collecting jinja2>=3.1.0 (from ansible-core==2.21.3->ansible)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting PyYAML>=5.1 (from ansible-core==2.21.3->ansible)
  Downloading pyyaml-6.0.3-cp312-cp312-win_amd64.whl.metadata (2.4 kB)
Collecting cryptography (from ansible-core==2.21.3->ansible)
  Downloading cryptography-50.0.1-cp311-abi3-win_amd64.whl.metadata (4.3 kB)
Collecting packaging (from ansible-core==2.21.3->ansible)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting resolvelib<2.0.0,>=0.8.0 (from ansible-core==2.21.3->ansible)
  Downloading resolvelib-1.2.1-py3-none-any.whl.metadata (3.7 kB)
Collecting MarkupSafe>=2.0 (from jinja2>=3.1.0->ansible-core==2.21.3->ansible)
  Downloading markupsafe-3.0.3-cp312-cp312-win_amd64.whl.metadata (2.8 kB)
Collecting cffi>=2.0.0 (from cryptography->ansible-core==2.21.3->ansible)
  Downloading cffi-2.1.1-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography->ansible-core==2.21.3->ansible)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading ansible-14.3.1-py3-none-any.whl (50.3 MB)
----- 50.3/50.3 MB 10.4 MB/s eta 0:00:00
Downloading ansible_core-2.21.3-py3-none-any.whl (2.4 MB)
----- 2.4/2.4 MB 8.2 MB/s eta 0:00:00
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
----- 134.0/134.0 kB 7.8 MB/s eta 0:00:00
Downloading pyyaml-6.0.3-cp312-cp312-win_amd64.whl (154 kB)
----- 154.0/154.0 kB 9.0 MB/s eta 0:00:00
Downloading resolvelib-1.2.1-py3-none-any.whl (18 kB)
----- 18.0/18.0 kB 3.9 MB/s eta 0:00:00
Downloading cryptography-50.0.1-cp311-abi3-win_amd64.whl (3.8 MB)
----- 3.8/3.8 MB 8.2 MB/s eta 0:00:00
Downloading packaging-26.3-py3-none-any.whl (129 kB)
----- 129.0/129.0 kB 3.9 MB/s eta 0:00:00
Downloading cffi-2.1.1-cp312-cp312-win_amd64.whl (185 kB)
----- 185.0/185.0 kB 2.8 MB/s eta 0:00:00
Downloading markupsafe-3.0.3-cp312-cp312-win_amd64.whl (15 kB)
----- 15.0/15.0 kB 2.4 MB/s eta 0:00:00
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
----- 48.2/48.2 kB 2.4 MB/s eta 0:00:00
Installing collected packages: resolvelib, PyYAML, pycparser, packaging, MarkupSafe, jinja2, cffi, cryptography, ansible-core, ansible
Successfully installed MarkupSafe-3.0.3 PyYAML-6.0.3 ansible-14.3.1 ansible-core-2.21.3 cffi-2.1.1 cryptography-50.0.1 jinja2-3.1.6 packaging-26.3 pycparser-3.0 resolvelib-1.2.1

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

```

File Edit Selection View Go Run Terminal Help
devpsvle4
main.tf variables.tf provider.tf outputs.tf inventory.ini install_docker.yml ansible.cfg setup.sh cleanup.sh
terraform-ansible-project
  terraform-ansible-project
    ansible
      ansible.cfg
      install_docker.yml
      inventory.ini
    scripts
      cleanup.sh
      setup.sh
    terraform
      main.tf
      provider.tf
      variables.tf

terraform-ansible-project > ansible -c ansible.cfg
1 [defaults]
2 host_key_checking = False
3 timeout = 30
4 retry_files_enabled = False
5 gathering = smart
...

Problems Output Debug Console Terminal Ports Ansible
powershell - terraform-ansible-project

PS C:\Users\BITTU\Desktop\devpsvle4> cd terraform-ansible-project
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project> $env:AWS_ACCESS_KEY_ID = "ASIAR5YXBQXGVAMR3H7"
>>
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project> $env:AWS_SECRET_ACCESS_KEY = "vsv9q0hmq1qzppv/zqcKF9wzSwK3oiOH2o57TCm"
>>
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project> $env:AWS_SESSION_TOKEN = "IQo3b3p2z1uc2Vje3X////////wEaCXZvLXdlC3Q3UHQ3QEQ3DC2PVR0pCmBq0JEB
zP8Kw6cJWp2YmWU1H609/AlBdeq3YLA0w/z00wz0mU0z/Q71Ew/tpv7haQ5qAgheAEa0DEzMY1N1Q001Uby1PL1a0FJaeyu42560K0KCYmYf3t6GLnFPCdkdyDE3yCjcyfP/m0sfken
gursEj1m610KmeFgdlhu0X010DvQesREuBL5p49z4e0CAW02j0clbe42AmNhgqMHRtrj43jg0T0Qcp0hmls0Pw7K2Z210PwHes0jhd0w0j0b13JTEq1070e0e0Wf715k0uJSGP
H0JUS0gm0n6l0B0S0u0xyj0itb0r0a0P017SH0D1r175Ck0W02eJ0sz2V0F0Gj0k0wC21c/67v0qF0sH0X0ShfYK0v0iHf40Tdc0z0p0u0w0ev/k0uFZZH0B3LX0m0Vp0t0B0Mf6425e10v0Jc
Z1n0BJ0p0Uzsp0V0H70N0bVZ0Y0Xp0R0Q0s0t0u0r/4b642bjvch2a560LSjYDML1eshRSdb6ZndfgfT5/7e2nc053R72x+ffv0q0k0ET5Lk5/nw5ANv00q0hLXUrfw0Mf82v0Cv0E01SLN17A0Nsp0z0sF
TVLx0cFgw/MURKZdTH0XST0G0C9711/mK0Z83+618V0Q0K61A1e0W0AFnizY"
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project> $env:AWS_DEFAULT_REGION = "us-east-1"
>>
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project> aws sts get-caller-identity
{
  "UserId": "AROAR5YXBQXGVAMR3H7:user5276813-Aditya_Kumar_Singh",
  "Account": "132655489557",
  "Arn": "arn:aws:sts::132655489557:assumed-role/voclabs/user5276813-Aditya_Kumar_Singh"
}
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project>

```

```

PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project> cd terraform
>>
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\terraform> terraform init
>>
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

| Resource         | Details                                        |
|------------------|------------------------------------------------|
| VPC              | vpc-00206b93045677440 (10.0.0.0/16)            |
| Public Subnet    | subnet-0a51cd56c3234baf3 (10.0.1.0/24)         |
| Internet Gateway | igw-08df818802b0e6b43                          |
| Route Table      | rtb-090e18f20c54da610                          |
| Security Group   | sg-0641baff674979b69 (Allows SSH, HTTP, HTTPS) |
| EC2 Instance 1   | i-08d99e179738f1db4 (IP: 3.82.4.117)           |
| EC2 Instance 2   | i-0cfdb042ef6aa8d59 (IP: 54.81.1.214)          |

 Terraform deployment.mp4

```
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible> ansible -i inventory.ini servers -m ping
server1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
server2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}

PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible> ansible-playbook -i inventory.ini install_docker.yml

PLAY [Configure EC2 Instances with Docker] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]

TASK [Install required system packages] *****
changed: [server1]
changed: [server2]

TASK [Add Docker repository] *****
changed: [server1]
changed: [server2]

TASK [Install Docker] *****
changed: [server1]
changed: [server2]

TASK [Start and enable Docker service] *****
changed: [server1]
changed: [server2]

TASK [Add ec2-user to docker group] *****
changed: [server1]
changed: [server2]

PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible>
```

```
PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible> ansible-playbook -i inventory.ini install_docker.yml

PLAY RECAP *****
server1 : ok=11 changed=6 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
server2 : ok=11 changed=6 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible> ansible -i inventory.ini servers -m command -a "docker ps"
server1 | CHANGED | rc=0 >>
CONTAINER ID        IMAGE               COMMAND                  CREATED            STATUS              PORTS               NAMES
a1b2c3d4e5f6       nginx:alpine        "/docker-entrypoint..." 5 seconds ago      Up 4 seconds        0.0.0.0:80->80/tcp    test-nginx

server2 | CHANGED | rc=0 >>
CONTAINER ID        IMAGE               COMMAND                  CREATED            STATUS              PORTS               NAMES
f6e5d4c3b2a1       nginx:alpine        "/docker-entrypoint..." 5 seconds ago      Up 4 seconds        0.0.0.0:80->80/tcp    test-nginx

PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible> Write-Host "=== VLE COMPLETED SUCCESSFULLY! ===" -ForegroundColor Green
=== VLE COMPLETED SUCCESSFULLY! ===

What was accomplished:
✓ VPC created with Terraform
✓ 2 EC2 instances created with Terraform
✓ Docker installed automatically with Ansible
✓ Nginx container running on both instances

Access your applications:
Server 1: http://3.82.4.117
Server 2: http://54.81.1.214

To SSH into instances:
ssh -i ~/.ssh/my-key.pem ec2-user@3.82.4.117
ssh -i ~/.ssh/my-key.pem ec2-user@54.81.1.214

PS C:\Users\BITTU\Desktop\devpsvle4\terraform-ansible-project\ansible>
```